information

MDPI

Article

Random Forest-Based Network Intrusion Detection with Feature Selection and Class Balancing on UNSW-NB15 Traffic

Jiří Pospíchal *, Aleš Augustín, Ladislav Huraj, Peter Střelec and Darja Gabriška

Institute of Computer Technologies and Informatics, Faculty of Natural Sciences, University of St. Cyril and Methodius in Trnava, 91701 Trnava, Slovakia; augustin2@ucm.sk (A.A.); ladislav.huraj@ucm.sk (L.H.); peter.strelec@ucm.sk (P.S.); darja.gabriska@ucm.sk (D.G.)

* Correspondence: jiri.pospichal@ucm.sk; Tel.: +421-944-386-057

Abstract

Machine-learning intrusion detection is challenged by attacks resembling legitimate traffic and by class imbalance. This study evaluates Random Forest detection on the UNSW-NB15 dataset for five binary attack-versus-normal tasks: DoS, Exploit, Backdoor, Analysis, and Reconnaissance. Categorical attributes were label-encoded, features selected using mutual information, hyperparameters optimized by randomized search, decision thresholds tuned on validation data, and SMOTE applied to training data at a fixed target of 56,000 minority samples. SVM and KNN results are descriptive because the shared label-encoded representation precludes model-independent comparison. Without SMOTE, Random Forest achieved F1-scores of 0.9468 for DoS, 0.9504 for Exploit, 0.9839 for Backdoor, 0.9317 for Analysis, and 0.9720 for Reconnaissance. Fixed-SMOTE pipelines achieved 0.9539, 0.9516, 0.9668, 0.8370, and 0.9651, respectively. Thus, SMOTE improved DoS, marginally improved Exploit, and reduced performance for Backdoor, Analysis, and Reconnaissance. Confusion matrices showed fewer false positives and false negatives for DoS, while Analysis gained recall but produced substantially more false positives. These results compare separately optimized complete pipelines and do not isolate SMOTE's effect. They characterize filtered target-attack-versus-normal streams, not operational mixed-attack traffic. Under this protocol, performance and precision–recall trade-offs were attack-dependent under the evaluated fixed-SMOTE experimental configuration, indicating that oversampling should be evaluated separately for each attack category.

Keywords: random forest; intrusion detection; network security; UNSW-NB15; SMOTE; feature selection; machine learning; cyberattack classification; SVM; KNN

Check for updates

Academic Editors: Jairo A. Gutierrez, Pavlos Papadopoulos, Nikolaos Pitropakis and Sokratis Katsikas

Received: 2 July 2026
Revised: 7 August 2026
Accepted: 27 August 2026
Published: 28 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Cybersecurity systems must increasingly detect attacks in network traffic that is large-scale, noisy, heterogeneous, and imbalanced. Anomaly-based intrusion detection has therefore become an important research direction, particularly because conventional signature-based systems are less effective against novel, adaptive, or obfuscated attacks [1]. Machine learning has been widely investigated for intrusion detection because it can learn statistical regularities from labeled traffic and classify unseen network flows [2]. Recent surveys also show that machine-learning-based IDSs are increasingly important for protecting critical infrastructure and IoT-enabled environments, where attack surfaces are expanding rapidly [3,4]. From an information systems perspective, intrusion detection can be viewed as a high-dimensional decision-support problem in which heterogeneous

network-flow records must be transformed into reliable security-relevant information under uncertainty and class imbalance.

Network intrusion detection is particularly challenging because different attacks exhibit distinct statistical profiles. Denial-of-Service (DoS) attacks may be associated with abnormal traffic intensity; Exploit attacks may exploit vulnerabilities in protocols or applications; Backdoor traffic may be rare but security-critical; Reconnaissance traffic may resemble legitimate probing or service discovery; and Analysis attacks may combine scanning, web-oriented probes, and malicious content delivery. The UNSW-NB15 dataset is suitable for studying such problems because it contains modern normal and malicious traffic and has been statistically analyzed as a benchmark for network anomaly detection [5]. Recent studies using UNSW-NB15 further confirm its value for evaluating machine-learning-based intrusion detection and class imbalance handling [6,7]. More broadly, benchmark dataset design remains a central issue in network intrusion detection research because dataset structure, feature representation, and attack taxonomy strongly influence reported model performance [8].

This study evaluates Random Forest classifiers for five UNSW-NB15 attack categories: DoS, Exploit, Backdoor, Analysis, and Reconnaissance. Each category is modeled as a separate attack-versus-normal binary classification task, enabling attack-specific analysis of feature selection, threshold tuning, and synthetic oversampling without obscuring minority-class behavior in an aggregate multiclass score. The evaluation framework also includes KNN and SVM baselines, mutual-information-based feature selection, and imbalance-aware metrics [9–24].

The primary contribution is an attack-specific comparison of separately optimized Random Forest pipelines trained on the original data and with a fixed SMOTE target of 56,000 minority-class samples. Across five UNSW-NB15 attack categories, confusion-matrix analysis identifies the error mechanisms underlying the observed F1-score differences: reductions in both false positives and false negatives for DoS, a precision–recall trade-off for Exploit, and a substantial increase in false positives for Analysis. Backdoor achieved its highest F1-score without oversampling despite having the fewest original training samples, showing that minority-class size alone did not predict which pipeline performed better.

Accordingly, balancing strategies are evaluated separately for each attack category within the Random Forest pipeline.

The remainder of the paper is organized as follows. Section 2 reviews related work and background. Section 3 describes the dataset, preprocessing pipeline, model design, feature selection, hyperparameter optimization, and evaluation metrics. Section 4 presents the experimental results. Section 5 discusses the implications of the findings. Section 6 concludes the paper.

2. Related Work

Machine-learning-based intrusion detection has been widely studied as a response to the limitations of rule-based and signature-based detection systems [1,2]. Recent surveys emphasize that ML-based IDSs are increasingly relevant for critical infrastructure, IoT, and heterogeneous cyber-physical environments [3,4]. At the same time, prior work has cautioned that intrusion detection differs from many standard classification tasks because traffic distributions are dynamic, labeled data are imperfect, and operational false positives are costly [2]. Although the present study is empirical, it is broadly consistent with formal perspectives that view computational processes as transformations of system states into interpretable outcomes [25].

The UNSW-NB15 dataset has become a frequently used benchmark because it was designed to include modern normal traffic patterns and multiple attack categories. Statistical

analysis of UNSW-NB15 has shown that it is more complex than older benchmark datasets and better suited for contemporary network anomaly detection evaluation [5]. Recent work has used UNSW-NB15 to compare classical machine-learning algorithms and has reported strong performance for Random Forest-based detection [6]. Other studies have examined oversampling and imbalance handling on UNSW-NB15, showing that preprocessing choices can significantly affect classifier performance [7]. Dataset surveys also show that differences in recording environment, feature representation, labeling, and attack taxonomy can substantially affect the evaluation of network intrusion detection systems [8].

Random Forest is an ensemble classifier introduced by Breiman that combines multiple decision trees trained using bootstrap sampling and randomized feature selection [9]. Large-scale comparisons of supervised learning algorithms have shown that ensemble tree methods are among the strongest general-purpose classifiers for tabular data [10]. In intrusion detection, this is particularly relevant because network traffic contains nonlinear relationships, mixed numerical and categorical attributes, and local threshold effects. KNN and SVM provide useful baselines: KNN relies on local neighborhood structure in feature space [11], while SVM constructs margin-based decision boundaries and can model nonlinear separability through kernel functions [12].

Class imbalance is a persistent problem in intrusion detection because some attack classes occur far less frequently than normal traffic or dominant attack categories. SMOTE addresses this problem by generating synthetic minority-class samples through interpolation between nearest minority neighbors [13]. Broader studies of imbalanced learning show that oversampling, cost-sensitive learning, and ensemble methods can improve minority-class detection, but they may also introduce false positives if synthetic examples overlap with majority-class regions [14–16].

The methodology used in this study is grounded in standard supervised learning and feature selection practice. The implementation relies on scikit-learn, a widely used Python machine-learning library [17]. Feature selection is motivated by the broader literature on variable selection and dimensionality reduction [18], including mutual-information-based feature ranking [19,20] and wrapper-based subset selection principles [21]. Model performance is evaluated using accuracy, precision, recall, and F1-score, which are standard metrics for binary classification and imbalanced detection tasks [22–24].

The dataset context also connects this work to earlier intrusion-detection benchmarks. The 1999 DARPA evaluation remains an important historical reference for systematic IDS evaluation [26]. More recent work extends IDS evaluation to IoT environments, ensemble-based statistical flow features, class overlap, explainable machine learning, and dataset heterogeneity [27–32]. These studies reinforce the need for reproducible evaluation pipelines and careful interpretation of model performance across different feature spaces and attack taxonomies.

Existing studies have established UNSW-NB15, Random Forest classifiers, and oversampling as relevant components of intrusion detection research. However, many evaluations report aggregate binary or multiclass performance and therefore provide limited insight into how oversampling affects individual attack categories. This study addresses that gap by isolating five attack types and evaluating the effect of SMOTE separately for each detector.

Recent IDS research includes explainable methods, boosting and stacked ensembles, graph neural networks, Transformers, and federated learning [33–37]. These methods provide context for current developments in transparency, relational and temporal representation, and distributed learning, but they are not experimental baselines in this study. The present analysis is limited to attack-specific balancing within the implemented Random Forest pipeline. Prior work has established that oversampling can either improve or

degrade performance in imbalanced classification. Building on this foundation, the present study contributes an attack-specific, confusion-matrix-level comparison of separately optimized Random Forest pipelines with and without a fixed SMOTE setting. The analysis distinguishes changes associated with false positives, false negatives, and precision–recall trade-offs.

3. Materials and Methods

3.1. Dataset

Experiments were conducted using the UNSW-NB15 dataset, a labeled network traffic dataset containing both normal and malicious records. The dataset has been statistically analyzed as a modern intrusion detection benchmark and compared with older network anomaly detection datasets [5]. It has also been used in recent machine-learning-based IDS studies involving Random Forest classifier, SVM, decision trees, and oversampling techniques [6,7].

The official UNSW-NB15 training partition was used for model fitting and hyperparameter search. The official test partition was divided once into a validation subset for threshold selection and a held-out evaluation subset for final reporting, as summarized in Table 1.

Table 1. Dataset partitions used in the experiments.

Partition	Number of Records	Purpose
Training set	175,341	Model training and hyperparameter search
Test partition	82,332	Split into validation and final test subsets
Validation subset of test partition	30% of test partition	Decision-threshold tuning
Held-out evaluation subset	70% of test partition	Final model evaluation

The attack categories investigated in this paper were DoS, Exploit, Backdoor, Analysis, and Reconnaissance. Each experiment was formulated as a binary task in which one attack category was labeled positive and normal traffic was labeled negative; all non-target attack categories were excluded. The reported metrics therefore characterize target-attack-versus-normal separation rather than performance on a mixed-attack stream [14].

This filtered design removes attack-to-attack confusion and is analytical rather than operational: false positives represent normal records classified as the target attack, not non-target attacks assigned to the wrong detector.

Table 2 reports the original training-set counts for the five evaluated attack categories.

Table 2. Attack categories and original training-set counts.

Attack Category	Original Training Samples
DoS	12,264
Exploit	33,393
Backdoor	1746
Analysis	2000
Reconnaissance	10,491

3.2. Preprocessing

The preprocessing pipeline consisted of four main stages:

1. Binary filtering: for each experiment, only normal traffic and the selected attack category were retained.

2. Categorical encoding: categorical features such as proto, service, and state were transformed into numerical representations using label encoding.

3. Train/validation/test organization: the original training split was used for model fitting, and the original test split was divided into validation and final test subsets while preserving the binary task structure for each attack category.

4. Optional class balancing: SMOTE was applied only to the training data in experiments designed to evaluate oversampling [13], not to the test data, and it was applied before feature selection.

Categorical variables were label-encoded using a common representation across classifiers. Because integer encoding can impose artificial ordering on nominal variables, it may affect KNN and SVM more strongly than Random Forest. This representational limitation is considered when interpreting the baseline comparison.

The implementation used Python libraries including pandas, numpy, scikit-learn, and imbalanced-learn. The machine-learning workflow followed standard model-selection and preprocessing practices supported by scikit-learn [17].

3.3. Feature Selection

Feature selection was performed using SelectKBest with mutual information as the scoring function. Feature selection is widely used to improve model efficiency, reduce irrelevant variables, and improve interpretability [18]. Mutual information is suitable for identifying nonlinear dependencies between candidate features and target labels [19,20].

The number of selected features k was varied over the following values:

$$k \in \{5, 10, 15, 20, 25, 30, 35\}.$$

For each value of k, the selected feature subset was used to train and evaluate candidate models. The selected number of features varied across attack categories, indicating that different subset sizes were preferred under the reported optimization procedure. This result is consistent with evidence that a single feature-subset size need not be optimal across classification tasks [18,21].

In the SMOTE condition, mutual-information feature selection followed oversampling, allowing synthetic observations to influence the selected feature subset. The analysis therefore compares complete, separately optimized pipelines rather than isolating SMOTE while holding the feature representation constant.

3.4. Classification Models

3.4.1. Random Forest

Random Forest was the primary classifier. It constructs an ensemble of decision trees, each trained on a bootstrap sample of the data and using randomized feature subsets during splitting [9]. For classification, the final prediction is obtained by aggregating tree votes. Random Forest classifier was selected because it is robust for tabular data, handles nonlinear interactions, is relatively resistant to overfitting, and performs well on heterogeneous network features [9,10].

The optimized Random Forest hyperparameters included n_estimators, min_samples_leaf, max_features, and class_weight.

3.4.2. Support Vector Machine

SVM with an RBF kernel was used as a baseline classifier. SVM constructs separating hyperplanes and can handle nonlinear classification through kernel functions [12]. The optimized hyperparameters were C, controlling regularization, and γ, controlling the influence of individual samples in the RBF kernel.

3.4.3. K-Nearest Neighbors

KNN was used as a second baseline. The nearest-neighbor decision rule assigns labels based on the closest labeled samples in the feature space [11]. The optimized hyperparameters were n_neighbors, weights, and metric. KNN provides a simple instance-based comparison but can be sensitive to irrelevant features and computationally expensive at prediction time.

3.5. Hyperparameter Optimization and Threshold Tuning

Hyperparameters were optimized using RandomizedSearchCV. Unlike exhaustive grid search, randomized search samples a fixed number of parameter combinations, reducing computational cost while still exploring a broad search space. The same search procedure was applied per attack category and per feature subset. This procedure was implemented using scikit-learn model-selection tools [17].

For each attack category and each selected value of k, hyperparameter search was conducted on the training data, threshold selection was performed on the validation subset, and the final metrics were computed only on the held-out final test subset. SMOTE, when used, was applied only to the training data to avoid information leakage into validation or final evaluation.

After model training, the decision threshold was tuned on the validation subset to maximize F1-score. This step is important in imbalanced binary classification because the default threshold of 0.5 may not provide the best trade-off between precision and recall [22–24]. The final scores were reported on the held-out final test subset.

3.6. SMOTE-Based Class Balancing

SMOTE was applied to the training data to address class imbalance by generating synthetic minority-class samples [13]. For each minority instance, synthetic examples are created by interpolating between the instance and its nearest minority-class neighbors. Studies of imbalanced learning show that oversampling can improve minority-class detection, but the benefit depends on class separability and overlap between minority and majority regions [14–16]. For every attack category, SMOTE increased the minority class to a fixed target of 56,000 training observations. Because this target was not optimized or subjected to sensitivity analysis, the reported results apply only to this oversampling level.

3.7. Evaluation Metrics

The models were evaluated using accuracy, precision, recall, and F1-score. These metrics are widely used for classification evaluation, particularly where false positives and false negatives have different practical implications [22–24]. For binary classification, the confusion matrix consists of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

Accuracy is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (1)$$

Precision is defined as:

$$Precision = \frac{TP}{TP + FP}. \quad (2)$$

Recall is defined as:

$$Recall = \frac{TP}{TP + FN}. \quad (3)$$

F1-score is defined as:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (4)$$

F1-score was treated as the primary optimization metric because it balances false positives and false negatives, which is especially relevant in intrusion detection [22–24].

3.8. Implementation Details

Experiments were run on a Windows 11 workstation with an Intel Core i7 processor and 32 GB RAM. The pipeline used Python 3.10.12, scikit-learn 1.3.0, imbalanced-learn 0.11.0, pandas 2.0.3, and NumPy 1.24.3. Stochastic procedures used random_state = 10, and supported computations used n_jobs = −1. Source code and data links are provided in the Supplementary Materials statement.

4. Results

The following results characterize target-attack-versus-normal classification on attack-filtered test streams and are interpreted within that task definition.

4.1. Random Forest Classifier Performance Without SMOTE

Table 3 summarizes the best Random Forest classifier results obtained for each attack category without SMOTE, while Table 4 reports the corresponding hyperparameter settings selected by randomized search. The optimization was performed using RandomizedSearchCV with n_iter = 10, cv = 3, scoring = f1, and random_state = 10; the same search protocol was later applied to the SVM and KNN baselines to ensure methodological consistency. The search space included n_estimators ∈ {100, 200, 300, 400}, min_samples_leaf ∈ {1, 2, 3}, max_features ∈ {None, sqrt}, and class_weight ∈ {None, balanced}. The number of selected features, k, was optimized separately from the set {5, 10, 15, 20, 25, 30, 35}, with the resulting best values reported in Table 3.

Table 3. Best Random Forest classifier results without SMOTE.

Attack Category	Accuracy	Precision	Recall	F1-Score	Best k
DoS	0.9896	0.9682	0.9263	0.9468	15
Exploit	0.9769	0.9462	0.9546	0.9504	35
Backdoor	0.9995	0.9975	0.9706	0.9839	25
Analysis	0.9976	0.9747	0.8924	0.9317	35
Reconnaissance	0.9952	0.9865	0.9579	0.9720	35

Table 4. Optimized hyperparameters for best Random Forest classifier results without SMOTE.

Attack Category	n_estimators	min_samples_leaf	max_features	class_weight
DoS	100	3	None	None
Exploit	400	3	None	None
Backdoor	100	3	None	None
Analysis	100	3	None	None
Reconnaissance	100	3	None	None

All five non-SMOTE models selected min_samples_leaf = 3 and max_features = None. Thus, the selected configurations used the same minimum leaf size and allowed each split to consider the full feature subset retained for that attack category.

The selected ensemble size was 400 trees for Exploit and 100 trees for the other four categories.

All five non-SMOTE models selected class_weight = None from the sampled configurations. Thus, explicit class weighting was not part of the best configuration identified for any attack category under the reported search protocol.

The strongest F1-score was obtained for Backdoor detection, followed by Reconnaissance, Exploit, DoS, and Analysis. Although Backdoor had the fewest original training samples, its non-SMOTE pipeline obtained precision of 0.9975 and recall of 0.9706. This result shows that low minority-class frequency did not preclude strong target-versus-normal separation under the evaluated protocol.

The Analysis category achieved the lowest F1-score among the non-SMOTE Random Forest classifier results. Its recall was lower than its precision, indicating that the model was conservative when predicting Analysis attacks and missed a larger fraction of actual positive cases.

4.2. Random Forest Classifier Performance with SMOTE

Table 5 reports the best Random Forest classifier results after applying SMOTE to the training data, and Table 6 gives the corresponding hyperparameter settings selected from the same search space used in the non-SMOTE experiments. Compared with Table 4, the selected settings show that oversampling changed the preferred model configuration for several attack categories. In particular, class_weight = balanced was selected for Exploit, Backdoor, Analysis, and Reconnaissance, suggesting that the synthetic expansion of the minority class did not fully remove the need for imbalance-aware learning in these cases.

Table 5. Best Random Forest classifier results with SMOTE.

Attack Category	Accuracy	Precision	Recall	F1-Score	Best *k*
DoS	0.9910	0.9717	0.9368	0.9539	30
Exploit	0.9776	0.9515	0.9516	0.9516	35
Backdoor	0.9990	0.9704	0.9632	0.9668	10
Analysis	0.9935	0.7635	0.9262	0.8370	20
Reconnaissance	0.9941	0.9806	0.9501	0.9651	30

Table 6. Optimized hyperparameters for best Random Forest classifier results with SMOTE.

Attack Category	n_estimators	min_samples_leaf	max_features	class_weight
DoS	400	1	sqrt	None
Exploit	200	3	None	balanced
Backdoor	100	2	None	balanced
Analysis	300	2	None	balanced
Reconnaissance	100	2	None	balanced

Compared with the non-SMOTE configurations, four of the five SMOTE-based pipelines selected smaller minimum leaf sizes. The DoS pipeline also selected the largest ensemble, with 400 trees, and was the only configuration to use max_features = sqrt; the remaining categories used max_features = None. By contrast, for Backdoor, Analysis, and Reconnaissance, SMOTE did not improve the final F1-score despite altered hyperparameter settings, reinforcing the conclusion that oversampling must be evaluated empirically for each attack category rather than applied as a default preprocessing step.

Relative to the separately optimized original-data pipelines, the fixed-SMOTE pipelines obtained a higher F1-score for DoS, a marginally higher value for Exploit, and lower values for Backdoor, Analysis, and Reconnaissance. The largest observed difference was for Analysis, where F1-score decreased from 0.9317 to 0.8370 as higher recall was accompanied by substantially lower precision.

4.3. Performance Comparison with and Without SMOTE

Table 7 compares the F1-scores of the optimized configurations trained with and without SMOTE.

Table 7. F1-score comparison with and without SMOTE.

Attack Category	F1, Optimized Original-Data Pipeline	F1, Optimized Fixed-SMOTE Pipeline	Difference	Higher F1 Under Evaluated Settings
DoS	0.9468	0.9539	+0.0071	SMOTE
Exploit	0.9504	0.9516	+0.0012	SMOTE
Backdoor	0.9839	0.9668	−0.0171	Original
Analysis	0.9317	0.8370	−0.0947	Original
Reconnaissance	0.9720	0.9651	−0.0069	Original

The fixed-SMOTE pipelines obtained higher F1-scores for DoS and Exploit, although the Exploit difference was only 0.0012. The original-data pipelines obtained higher values for Backdoor, Analysis, and Reconnaissance, with the largest difference observed for Analysis at 0.0947. Because Backdoor had the fewest original minority samples yet performed best without oversampling, minority-class size alone did not predict which pipeline achieved the higher F1-score.

4.4. Confusion Matrices for Detection of Attacks Without and with SMOTE

Table 8 reports confusion matrices for the best SMOTE and non-SMOTE configurations, showing how oversampling changed true positives, false negatives, and false positives for each attack category.

For DoS, the complete pipeline trained with SMOTE produced 2681 true positives, 181 false negatives, and 78 false positives, whereas the separately optimized non-SMOTE pipeline produced 2651 true positives, 211 false negatives, and 87 false positives. The SMOTE-based pipeline therefore obtained a DoS F1-score of 0.9539, which was 0.0071 higher than the F1-score of 0.9468 obtained by the non-SMOTE pipeline. In this held-out evaluation sample, the higher F1-score was associated with reductions in both false negatives and false positives.

For Exploit, the SMOTE-based pipeline produced fewer false positives than the non-SMOTE pipeline, decreasing the count from 423 to 378, but it also produced more false negatives, increasing the count from 354 to 377. Accordingly, precision increased from 0.9462 to 0.9515, while recall decreased from 0.9546 to 0.9516. The resulting F1-score was 0.9516 for the SMOTE-based pipeline and 0.9504 for the non-SMOTE pipeline, corresponding to a marginal numerical difference of 0.0012. This result reflects a precision–recall trade-off rather than a consistent reduction in both types of classification error.

For Backdoor, both pipelines obtained high scores, but the non-SMOTE pipeline produced the more favorable confusion matrix. It generated 396 true positives, one false positive, and 12 false negatives, whereas the SMOTE-based pipeline generated 393 true positives, 12 false positives, and 15 false negatives. Precision decreased from 0.9975 to 0.9704, recall decreased from 0.9706 to 0.9632, and F1-score decreased from 0.9839 to 0.9668. Thus, the non-SMOTE pipeline obtained a Backdoor F1-score 0.0171 higher than the SMOTE-based pipeline under the evaluated settings. This result is notable because Backdoor was the smallest attack class in the original training data, indicating that minority-class cardinality alone did not predict which pipeline configuration would obtain the higher F1-score.

Analysis exhibited the largest observed difference between the two pipeline configurations. The SMOTE-based pipeline increased the number of true positives from 423 to 439

and reduced false negatives from 51 to 35. These changes corresponded to an increase in recall from 0.8924 to 0.9262. However, false positives increased from 11 to 136, and precision decreased from 0.9747 to 0.7635. Consequently, the SMOTE-based pipeline obtained an F1-score of 0.8370, which was 0.0947 lower than the F1-score of 0.9317 obtained by the non-SMOTE pipeline. In this case, the improvement in recall did not compensate for the substantially higher false-positive count.

For Reconnaissance, the SMOTE-based pipeline produced 2325 true positives, 122 false negatives, and 46 false positives, compared with 2344 true positives, 103 false negatives, and 32 false positives for the non-SMOTE pipeline. Precision and recall both decreased, resulting in an F1-score of 0.9651 for the SMOTE-based pipeline and 0.9720 for the non-SMOTE pipeline. The non-SMOTE pipeline therefore obtained an F1-score 0.0069 higher and produced fewer false positives and false negatives under the evaluated settings.

Overall, the pipeline differences were attack-specific: DoS showed fewer false positives and false negatives under the fixed-SMOTE configuration, Exploit presented a near-neutral precision–recall trade-off, and Backdoor, Analysis, and Reconnaissance favored the original-data configurations. These results describe complete, separately optimized pipelines rather than the isolated effect of SMOTE.

Table 8. Confusion matrices for attack detection without and with SMOTE (with row normalized values in parentheses).

	without SMOTE		with SMOTE	
	Predicted Normal	Predicted DoS	Predicted Normal	Predicted DoS
True Normal	25,814 (99.66%)	87 (0.34%)	25,823 (99.70%)	78 (0.30%)
True DoS	211 (**7.37%**)	2651 (92.63%)	181 (6.32%)	2681 (93.68%)
	without SMOTE		with SMOTE	
	Predicted Normal	Predicted Exploit	Predicted Normal	Predicted Exploit
True Normal	25,477 (98.37%)	423 (1.63%)	25,522 (98.54%)	378 (1.46%)
True Exploit	354 (4.54%)	7439 (95.46%)	377 (4.84%)	7416 (95.16%)
	without SMOTE		with SMOTE	
	Predicted Normal	Predicted Backdoor	Predicted Normal	Predicted Backdoor
True Normal	25,900 (99.996%)	1 (0.004%)	25,899 (99.95%)	12 (0.05%)
True Backdoor	12 (2.94%)	396 (97.06%)	15 (3.68%)	393 (96.32%)
	without SMOTE		with SMOTE	
	Predicted Normal	Predicted Analysis	Predicted Normal	Predicted Analysis
True Normal	25,889 (99.96%)	11 (0.04%)	25,764 (99.47%)	136 (0.53%)
True Analysis	51 (10.76%)	423 (89.24%)	35 (7.38%)	439 (92.62%)
	without SMOTE		with SMOTE	
	Predicted Normal	Predicted Reconnaissance	Predicted Normal	Predicted Reconnaissance
True Normal	25,869 (99.88%)	32 (0.12%)	25,855 (99.82%)	46 (0.18%)
True Reconn.	103 (4.21%)	2344 (95.79%)	122 (4.99%)	2325 (95.01%)

4.5. Descriptive SVM and KNN Results Under the Shared Label-Encoded Representation

Table 9 reports SVM and KNN results under the same label-encoded, unscaled representation used for Random Forest. This representation introduces artificial numerical

relationships among nominal categories and is better suited to tree-based models than to distance- or kernel-based classifiers. The results are therefore descriptive and do not support a model-independent ranking. Table 10 presents the descriptive characteristics of the classifiers implemented under the adopted preprocessing protocol.

Table 9. Quantitative comparison of classifiers.

	F1-Score			**Recall**			**Precision**		
Attack Category	RF	SVM	KNN	RF	SVM	KNN	RF	SVM	KNN
DoS	0.9468	0.8910	0.9224	0.9263	0.8686	0.9200	0.9682	0.9146	0.9248
Exploit	0.9504	0.8817	0.9008	0.9546	0.9557	0.9322	0.9462	0.8184	0.8714
Backdoor	0.9839	0.7889	0.9505	0.9706	0.9020	0.9657	0.9975	0.7010	0.9359
Analysis	0.9317	0.8157	0.7171	0.8924	0.9008	0.9304	0.9747	0.7452	0.5833
Reconnaissance	0.9720	0.8041	0.9049	0.9579	0.9726	0.9600	0.9865	0.6853	0.8557

Note: This table does not constitute a methodologically neutral classifier benchmark. The categorical representation and absence of scaling disadvantage distance- and kernel-based classifiers. No general model ranking should be inferred.

Table 10. Descriptive characteristics of the implemented classifiers under the present preprocessing protocol.

Classifier	**Strengths**	**Limitations Observed**
RF	Highest F1-score and precision under the present label-encoded protocol	Requires tuning of feature count and ensemble parameters; results may benefit from tree-compatible representation
SVM	Strong nonlinear decision boundaries with RBF kernel	Label encoding and absence of scaling compromise kernel geometry; lower overall F1-score in the reported experiments; sensitive to C and γ
KNN	Simple baseline; competitive recall in some cases	Label encoding and the absence of scaling compromise distance geometry; sensitive to feature selection and dimensionality

Table 11 reports the optimized hyperparameters for the SVM and KNN baseline models without SMOTE. For SVM, the radial basis function kernel was used because it can model nonlinear decision boundaries in the selected feature space. The number of selected features, k, was optimized from {5, 10, 15, 20, 25, 30, 35}, while the regularization parameter C was selected from {0.1, 0.5, 1, 5, 10} and the kernel coefficient γ from {scale, auto}. The parameter C controls the trade-off between margin width and classification errors: larger values place greater emphasis on correctly classifying training samples, whereas smaller values impose stronger regularization. The γ parameter controls the locality of the RBF kernel and therefore determines how strongly individual training samples influence the decision surface.

Table 11. Optimized hyperparameters for KNN and SVM without SMOTE.

	SVM			**KNN**			
Attack Category	***k***	***C***	**γ**	***K***	**Weights**	**n_neighbors**	**Metric**
DoS	30	10	auto	15	distance	7	Euclidean
Exploit	5	5	scale	5	distance	5	Manhattan
Backdoor	30	1	auto	10	uniform	3	Manhattan
Analysis	5	10	scale	30	uniform	3	Manhattan
Reconnaissance	30	10	scale	5	uniform	7	Manhattan

The selected SVM settings varied substantially across attacks, indicating that the baseline model required attack-specific tuning to achieve competitive performance. The selected value of C was 10 for DoS, Analysis, and Reconnaissance and 1 for Backdoor, demonstrating that the preferred SVM settings varied across attack categories under the reported search space. The optimal number of selected features also differed, ranging from only five features for Exploit and Analysis to 30 features for DoS, Backdoor, and Reconnaissance. This variability reinforces the broader finding that individual attack categories are characterized by different feature dependencies and decision-boundary complexity.

For KNN, the optimized parameters included the number of neighbors, the distance-weighting scheme, and the distance metric. The number of neighbors was selected from {3, 5, 7, 9, 11}, the weighting strategy from {uniform, distance}, and the metric from {Euclidean, Manhattan}. Smaller values of n_neighbors, such as 3 or 5, allow the classifier to respond to local structure in the feature space, while larger values provide smoother but less locally sensitive decisions. Distance weighting was selected for DoS and Exploit, while uniform weighting was selected for Backdoor, Analysis, and Reconnaissance. The Manhattan metric was selected for four of the five categories.

Under the present representation, Random Forest obtained higher F1-scores and precision than the implemented SVM and KNN configurations. Because the preprocessing was not model-neutral, Table 9 provides a descriptive comparison only. A controlled benchmark would require model-appropriate categorical encoding, fold-specific scaling, comparable search budgets, and independently defined hyperparameter spaces.

4.6. Contextual Comparison with Selected UNSW-NB15 Studies

This section places the reported results in the context of selected UNSW-NB15 studies. Because the studies differ in preprocessing, data partitions, retained classes, balancing methods, and task formulation, the reported values are descriptive reference points rather than controlled comparisons.

For DoS, the fixed-SMOTE pipeline reported accuracy of 0.9910, compared with 0.9885 in [38]. Its accuracy, precision, recall, and F1-score were also numerically higher than the values reported in [39]. Because the studies differ in task formulation and preprocessing, these values provide contextual reference points rather than evidence of comparative superiority.

For Exploit, the present SMOTE configuration reported lower precision than [38] (0.9515 vs. 0.9731) but higher recall (0.9516 vs. 0.8219) and F1-score (0.9516 vs. 0.8911). These values indicate a different reported precision–recall balance, but the protocol differences preclude a direct performance ranking like in [38].

For Reconnaissance, the non-SMOTE RF configuration reported higher precision than the bagging result in [40] (0.987 vs. 0.963) but lower recall (0.958 vs. 0.993) and F1-score (0.972 vs. 0.978). These independently reported values provide contextual evidence of differing precision–recall profiles.

For Backdoor, the non-SMOTE RF configuration reported higher recall (0.9706 vs. 0.9654), F1-score (0.9839 vs. 0.9528), and precision (0.9975 vs. 0.9482) than [41]. However, the present experiment excluded non-target attacks, whereas [41] retained them, so the values are not directly comparable.

For Analysis-type attacks, no reference result using a sufficiently similar attack-specific evaluation protocol was identified. The Analysis results are therefore interpreted only within the internal SMOTE and non-SMOTE comparison conducted in this study.

Overall, the external results place the reported values within the range of selected UNSW-NB15 studies. Because these studies use different experimental protocols, the

internally controlled comparisons in Sections 4.1–4.5 remain the primary basis for the study's conclusions.

5. Discussion

The Random Forest pipelines obtained high F1-scores across the five attack-versus-normal tasks, with category-specific precision–recall profiles. The SVM and KNN results are specific to the shared label-encoded representation and do not constitute a model-neutral benchmark. Broader comparison should use model-appropriate preprocessing and include contemporary boosting, deep, temporal, and graph-based architectures.

The preferred feature-subset size varied by attack category. The non-SMOTE DoS pipeline selected 15 features, while the Exploit, Analysis, and Reconnaissance pipelines selected 35. Because complete feature rankings were not retained, these results characterize subset size but not the importance or stability of individual attributes.

The differences between the original-data and fixed-SMOTE pipelines were also attack-specific. The DoS pipeline obtained a higher F1-score under the fixed-SMOTE condition, Exploit changed only marginally, and Backdoor, Analysis, and Reconnaissance obtained higher values under the original-data condition. Analysis showed the clearest error trade-off: higher recall was accompanied by more false positives and a lower F1-score. The SMOTE results also show that class imbalance alone is not sufficient justification for oversampling. Backdoor had the fewest original training samples, yet the non-SMOTE Random Forest classifier achieved the highest F1-score among the evaluated categories. Thus, the smallest original minority class produced its highest F1-score without oversampling, indicating that class frequency alone did not predict the preferred pipeline configuration. Therefore, imbalance handling should be evaluated per attack type, using F1-score and the precision–recall trade-off rather than accuracy alone.

The mutual-information procedure used in this study is a filter-based feature-selection mechanism and should not be interpreted as an explanation of individual model predictions. Post hoc methods such as SHAP, permutation importance, and LIME could examine whether the two pipeline configurations rely on different traffic attributes. For the Analysis detector, such methods could help identify features associated with the observed increase in false positives [42].

The binary-per-attack formulation provides an analytical decomposition rather than a complete IDS architecture. A possible extension would combine the detectors through a shared preprocessing layer and an explicit aggregation policy for confidence scores, simultaneous alerts, and multilabel outputs. Such an architecture would also require explicit policies for unknown or unmodeled attacks, score calibration, and conflicts among detector outputs. Simultaneous positive outputs should not necessarily be interpreted as contradictions, because a flow may exhibit overlapping attack characteristics. However, the present study did not evaluate score calibration, conflict-resolution strategies, or performance in the presence of non-target background attacks. Evaluation on a complete, unfiltered test stream containing normal traffic and all attack categories is therefore required before the proposed arrangement can be considered an operationally validated IDS architecture.

The results are specific to the UNSW-NB15 benchmark, whose feature representation and attack taxonomy differ from older and newer intrusion detection datasets [5,8]. This context is important when interpreting absolute performance values. The reported results characterize the performance of the Random Forest pipelines under the UNSW-NB15 attack-filtered evaluation setting. Related studies on benchmark heterogeneity, IoT datasets, explainable IDSs, and modern IDS architectures reinforce the importance of evaluating models in relation to dataset structure and operational context [26–31].

The DoS, Exploit, and Backdoor results fall within ranges reported in selected UNSW-NB15 studies, but differences in feature engineering, data partitioning, balancing, and task formulation preclude claims of comparative competitiveness.

5.1. Limitations

5.1.1. External Validity

The findings are specific to the UNSW-NB15 traffic environment, feature representation, and attack taxonomy. The evaluation covers five labeled attack categories and does not assess unseen attack families. Consequently, the conclusions are limited to the evaluated categories and should not be interpreted as evidence of zero-day detection capability. Such claims would require an open-set, novelty-detection, temporal-holdout, or leave-one-attack-family-out design. Validation on additional datasets, including CICIDS2017, CSE-CIC-IDS2018, Bot-IoT, ToN_IoT, and Edge-IIoTset, is also needed to determine whether the observed attack-specific differences between the original-data and fixed-SMOTE pipelines generalize to other traffic environments.

5.1.2. Experimental Uncertainty

The reported differences are point estimates obtained from one held-out evaluation subset. Repeated-seed experiments, bootstrap confidence intervals, or paired statistical tests such as McNemar's test would be required to quantify uncertainty. This limitation is particularly important for the Exploit category, for which the F1-score difference between the two pipeline configurations was only 0.0012. Future evaluation should combine uncertainty analysis with harmonized testing across additional benchmark and operational datasets.

Complete feature lists and rankings were not retained for every configuration. The results therefore support conclusions about the preferred number of selected features, but not about the importance or stability of individual attributes. In addition, no post hoc explanation method was applied. Mutual-information-based feature selection identifies feature–target associations but does not explain how the fitted Random Forest combines features in individual predictions. Future studies should preserve feature rankings across repeated samples and evaluate explanation methods such as SHAP and permutation importance, including their stability in the presence of correlated network features.

5.1.3. Pipeline Attribution and SMOTE Configuration

The original-data and fixed-SMOTE conditions were optimized as complete pipelines, including feature selection, Random Forest hyperparameters, and decision thresholds. The observed differences therefore characterize the final pipeline configurations and do not isolate the independent effect of SMOTE. A factorial ablation would be required to attribute the differences to individual pipeline components.

The SMOTE target of 56,000 minority-class observations was fixed and non-optimized, and no sensitivity analysis was conducted across alternative oversampling ratios. The direction and magnitude of the reported differences are therefore conditional on this single oversampling level. The study does not identify an optimal SMOTE ratio for any attack category.

In the oversampled condition, SMOTE was applied before mutual-information feature selection. Synthetic observations therefore contributed to the estimation of feature–target dependence and may have influenced both the identities and the number of selected features. Moreover, the realism of the synthetic observations was not assessed in the original feature space. The conclusions consequently concern predictive performance under the evaluated pipeline configurations rather than the realism of the generated traffic records. Future factorial evaluation should compare feature selection before and after

oversampling, preserve selected feature identities across repeated samples, and assess multiple attack-specific oversampling ratios.

5.1.4. Representation and Baseline Fairness

Label encoding introduces artificial ordinal relationships among nominal variables such as protocol, service, and state. This distortion particularly affects distance-based and kernel-based classifiers and may also influence Random Forest split selection. The SVM and KNN results therefore provide a descriptive comparison under the shared representation rather than a model-neutral benchmark. Future comparisons should use model-appropriate representations, such as one-hot, target, or embedding-based encoding, together with scaling procedures applied within the cross-validation pipeline.

The study also does not provide a contemporary state-of-the-art benchmark. Modern boosting, deep, temporal, graph-based, and Transformer architectures were not evaluated under the same data partition, filtering protocol, feature representation, or optimization budget. Their inclusion in the literature review provides disciplinary context only. Controlled comparison with these model families remains an important direction for future work.

5.1.5. Operational Validity

The binary-per-attack design excluded non-target attacks from model development and evaluation. This filtering removes cross-attack confusion and produces cleaner decision boundaries than a mixed operational stream. The reported metrics therefore characterize target-attack-versus-normal separation and are likely optimistic relative to deployment conditions, where non-target attacks may activate incorrect or multiple detectors. Operational assessment requires evaluation on an unfiltered test stream and explicit policies for score calibration, alert aggregation, conflicting detector outputs, and unknown attacks.

The models were evaluated offline, so their behavior under concept drift, changing services, adversarial adaptation, and other live-network conditions remains unverified. The reported metrics represent controlled benchmark performance rather than evidence of long-term deployment robustness. Training time, inference latency, throughput, memory use, and model size were also not measured systematically. The study therefore does not establish suitability for real-time or resource-constrained deployment. These properties should be evaluated on fixed hardware and software platforms under representative traffic loads.

6. Conclusions

This study provides an attack-specific, confusion-matrix-level comparison of complete Random Forest pipelines trained on the original UNSW-NB15 data and with a fixed SMOTE target of 56,000 minority-class samples. Across five attack categories, the analysis quantifies how the pipeline configurations differed in F1-score, false positives, and false negatives, and shows that minority-class frequency alone did not predict which configuration performed better. Under the evaluated settings, the SMOTE-based DoS pipeline obtained a numerically higher F1-score and reduced both false positives and false negatives. Exploit showed only a marginal numerical F1-score difference, accompanied by increased precision and reduced recall. Backdoor, Analysis, and Reconnaissance obtained higher F1-scores without SMOTE. The Analysis result was especially informative because the SMOTE-based configuration increased recall but generated substantially more false positives. Backdoor achieved the strongest non-SMOTE F1-score despite having the fewest original attack samples, showing that minority-class cardinality alone did not predict which pipeline configuration would perform better.

The findings are specific to one oversampling target and to separately optimized complete pipelines. Because evaluation used attack-filtered binary streams, the reported metrics represent target-attack-versus-normal separation rather than operational mixed-attack performance.

The results support four practical recommendations. Oversampling should be validated separately for each attack detector using both F1-score and false-positive rate, and it may be unnecessary for rare but well-separated categories. Changes in balancing strategy should trigger joint revalidation of feature selection, hyperparameters, and decision thresholds. Systems combining multiple binary detectors should retain confidence scores and define explicit aggregation or multilabel policies. Finally, performance measured on attack-filtered streams should not be treated as expected performance on mixed operational traffic.

Supplementary Materials: The source code, together with the training and testing sets, is available at: https://github.com/Alesos/Random-Forest-Based-Network-Intrusion-Detection-with-Feature-Selection-and-Class-Balancing (accessed on 26 August 2026).

Author Contributions: Conceptualization, J.P. and A.A.; methodology, P.S.; software, A.A.; validation, A.A. and L.H.; formal analysis, P.S.; investigation, D.G. and P.S.; resources, D.G.; data curation, L.H.; writing—original draft preparation, A.A.; writing—review and editing, J.P.; visualization, J.P.; supervision, J.P.; project administration, D.G.; funding acquisition, L.H. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Scientific Grant Agency of the Ministry of Education, Science, Research and Sport of the Slovak Republic and the Slovak Academy of Sciences, grant number VEGA 1/0175/25, Applying machine learning and digital fingerprinting to improve detection and mitigation of DDoS attacks.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The experiments used the publicly available UNSW-NB15 dataset, https://research.unsw.edu.au/projects/unsw-nb15-dataset# (accessed on 27 May 2026). Derived results are reported in the article. Additional implementation details may be made available by the authors upon reasonable request.

Acknowledgments: During the preparation of this manuscript, the authors used generative AI assistance for language editing and translation. The authors reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

DoS	Denial of Service attack
UNSW-NB15	University of New South Wales network intrusion detection dataset
SMOTE	Synthetic Minority Over-sampling Technique
RF	Random Forest
SVM	Support Vector Machine
KNN	K-Nearest Neighbors

References

1. García-Teodoro, P.; Díaz-Verdejo, J.; Maciá-Fernández, G.; Vázquez, E. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Comput. Secur.* **2009**, *28*, 18–28. [CrossRef]
2. Buczak, A.L.; Guven, E. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Commun. Surv. Tutor.* **2015**, *18*, 1153–1176. [CrossRef]

3. Pinto, A.; Herrera, L.-C.; Donoso, Y.; Gutierrez, J.A. Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure. *Sensors* **2023**, *23*, 2415. [CrossRef] [PubMed]
4. Asharf, J.; Moustafa, N.; Khurshid, H.; Debie, E.; Haider, W.; Wahab, A. A review of intrusion detection systems using machine and deep learning in Internet of Things: Challenges, solutions and future directions. *Electronics* **2020**, *9*, 1177. [CrossRef]
5. Moustafa, N.; Slay, J. The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inf. Secur. J. Glob. Perspect.* **2016**, *25*, 18–31. [CrossRef]
6. More, S.; Idrissi, M.; Mahmoud, H.H.; Asyhari, A.T. Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. *Algorithms* **2024**, *17*, 64. [CrossRef]
7. Ahmed, H.A.; Hameed, A.; Bawany, N.Z. Network intrusion detection using oversampling technique and machine learning algorithms. *PeerJ Comput. Sci.* **2022**, *8*, e820. [CrossRef] [PubMed]
8. Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. A survey of network-based intrusion detection data sets. *Comput. Secur.* **2019**, *86*, 147–167. [CrossRef]
9. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
10. Fernández-Delgado, M.; Cernadas, E.; Barro, S.; Amorim, D. Do we need hundreds of classifiers to solve real world classification problems? *J. Mach. Learn. Res.* **2014**, *15*, 3133–3181.
11. Cover, T.; Hart, P. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* **1967**, *13*, 21–27. [CrossRef]
12. Cortes, C.; Vapnik, V. Support-vector networks. *Mach. Learn.* **1995**, *20*, 273–297. [CrossRef]
13. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: Synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **2002**, *16*, 321–357. [CrossRef]
14. He, H.; Garcia, E.A. Learning from imbalanced data. *IEEE Trans. Knowl. Data Eng.* **2009**, *21*, 1263–1284. [CrossRef]
15. Galar, M.; Fernández, A.; Barrenechea, E.; Bustince, H.; Herrera, F. A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches. *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.* **2012**, *42*, 463–484. [CrossRef]
16. Krawczyk, B. Learning from imbalanced data: Open challenges and future directions. *Prog. Artif. Intell.* **2016**, *5*, 221–232. [CrossRef]
17. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
18. Guyon, I.; Elisseeff, A. An introduction to variable and feature selection. *J. Mach. Learn. Res.* **2003**, *3*, 1157–1182. [CrossRef]
19. Battiti, R. Using mutual information for selecting features in supervised neural net learning. *IEEE Trans. Neural Netw.* **1994**, *5*, 537–550. [CrossRef]
20. Peng, H.; Long, F.; Ding, C. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Trans. Pattern Anal. Mach. Intell.* **2005**, *27*, 1226–1238. [CrossRef] [PubMed]
21. Kohavi, R.; John, G.H. Wrappers for feature subset selection. *Artif. Intell.* **1997**, *97*, 273–324. [CrossRef]
22. Sokolova, M.; Lapalme, G. A systematic analysis of performance measures for classification tasks. *Inf. Process. Manag.* **2009**, *45*, 427–437. [CrossRef]
23. Fawcett, T. An introduction to ROC analysis. *Pattern Recognit. Lett.* **2006**, *27*, 861–874. [CrossRef]
24. Powers, D.M.W. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *J. Mach. Learn. Technol.* **2011**, *2*, 37–63.
25. Kohútová, M.; Steingartner, W.; Novotný, S. A categorical model of state transformations in programming languages. *J. Appl. Math. Stat. Inform.* **2026**, *22*, 67–80. [CrossRef]
26. Lippmann, R.P.; Haines, J.W.; Fried, D.J.; Korba, J.; Das, K. The 1999 DARPA off-line intrusion detection evaluation. *Comput. Netw.* **2000**, *34*, 579–595. [CrossRef]
27. Moustafa, N.; Turnbull, B.; Choo, K.-K.R. An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of Internet of Things. *IEEE Internet Things J.* **2019**, *6*, 4815–4830. [CrossRef]
28. Zoghi, Z.; Serpen, G. Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. *Concurr. Comput. Pract. Exp.* **2024**, *36*, e8242. [CrossRef]
29. Mohale, V.Z.; Obagbuwa, I.C. Evaluating machine learning-based intrusion detection systems with explainable AI: Enhancing transparency and interpretability. *Front. Comput. Sci.* **2025**, *7*, 1520741. [CrossRef]
30. Booij, T.M.; Chiscop, I.; Meeuwissen, E.; Moustafa, N.; den Hartog, F.T.H. ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets. *IEEE Internet Things J.* **2022**, *9*, 485–496. [CrossRef]
31. Alkhater, N. A rigorous comparative study of supervised machine learning techniques for network anomaly detection: Empirical insights from the UNSW-NB15 dataset. *Computers* **2026**, *15*, 285. [CrossRef]
32. Alromaihi, N.; Rouached, M.; Akremi, A. Design and analysis of an effective architecture for machine learning based intrusion detection systems. *Network* **2025**, *5*, 13. [CrossRef]
33. Okeke, D.C. A Systematic Review of Machine Learning Techniques in Intrusion Detection Systems. *J. Cyber Secur.* **2026**, *8*, 319–356. [CrossRef]

34. Gyimah, N.K.; Akinie, R.; Mwakalonge, J.; Izison, B.; Mukwaya, A.; Ruganuza, D.; Sulle, M. An AutoML-based approach for network intrusion detection. In Proceedings of the IEEE SoutheastCon 2025, Concord, NC, USA, 22–30 March 2025; pp. 1177–1183.
35. Wang, Y.; Han, Z.; Du, Y.; Li, J.; He, X. BS-GAT: A network intrusion detection system based on graph neural network for edge computing. *Cybersecurity* **2025**, *8*, 27. [CrossRef]
36. Xi, C.; Wang, H.; Wang, X. A novel multi-scale network intrusion detection model with transformer. *Sci. Rep.* **2024**, *14*, 23239. [CrossRef] [PubMed]
37. Sun, S.; Sharma, P.; Nwodo, K.; Stavrou, A.; Wang, H. FedMADE: Robust federated learning for intrusion detection in IoT networks using a dynamic aggregation method. In *Proceedings of the International Conference on Information Security*; Springer Nature: Cham, Switzerland, 2024; pp. 286–306.
38. Kamarudin, I.E.; Razak, M.F.A.; Firdaus, A.; Jaya, M.I.; Ti Dun, Y. Performance Analysis on Denial of Service attack using UNSW-NB15 Dataset. In *Proceedings of the 2021 International Conference on Software Engineering & Computer Systems and 4th International Conference on Computational Science and Information Management (ICSECS-ICOCSIM), Pekan, Malaysia, 24–26 August 2021*; IEEE: Piscataway, NJ, USA, 2021; pp. 423–426. Available online: https://ieeexplore.ieee.org/document/9537040 (accessed on 8 January 2026).
39. Kolesárová, D. Prediction of Selected Attacks in Network Traffic. Master's Thesis, Technical University of Košice, Košice, Slovakia, 2025. Available online: https://opac.crzp.sk/?fn=detailBiblioFormChildK2TT7&sid=11DFBA30F566A03CA8E0E07AA9BF&seo=CRZP-detail-kniha (accessed on 21 April 2026).
40. Nguyen, V.Q.; Ngo, L.T.; Nguyen, V.H.; Nguyen, L.M.; Le-Khac, N.-A. A Deep Metric Learning Approach for Cyber Reconnaissance Detection. In Proceedings of the 2024 1st International Conference on Cryptography and Information Security (VCRIS), Hanoi, Vietnam, 3–4 December 2024. Available online: https://ieeexplore.ieee.org/document/10813453 (accessed on 8 January 2026).
41. Sharma, A.; Babbar, H. Unveiling Backdoor Attacks in Networking with Machine Learning Approaches. In Proceedings of the 2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI), Coimbatore, India, 28–30 August 2024. Available online: https://ieeexplore.ieee.org/document/10696089 (accessed on 8 January 2026).
42. Mia, M.; Pritom, M.M.A.; Islam, T.; Hasan, K. Visually analyze SHAP plots to diagnose misclassifications in ML-based intrusion detection. In *Proceedings of the 2024 IEEE International Conference on Data Mining Workshops (ICDMW), Abu Dhabi, United Arab Emirates, 9–12 December 2024*; IEEE: Piscataway, NJ, USA, 2024; pp. 632–641.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.